Elektrotechnik Abgabe

Verwendete Bauteile

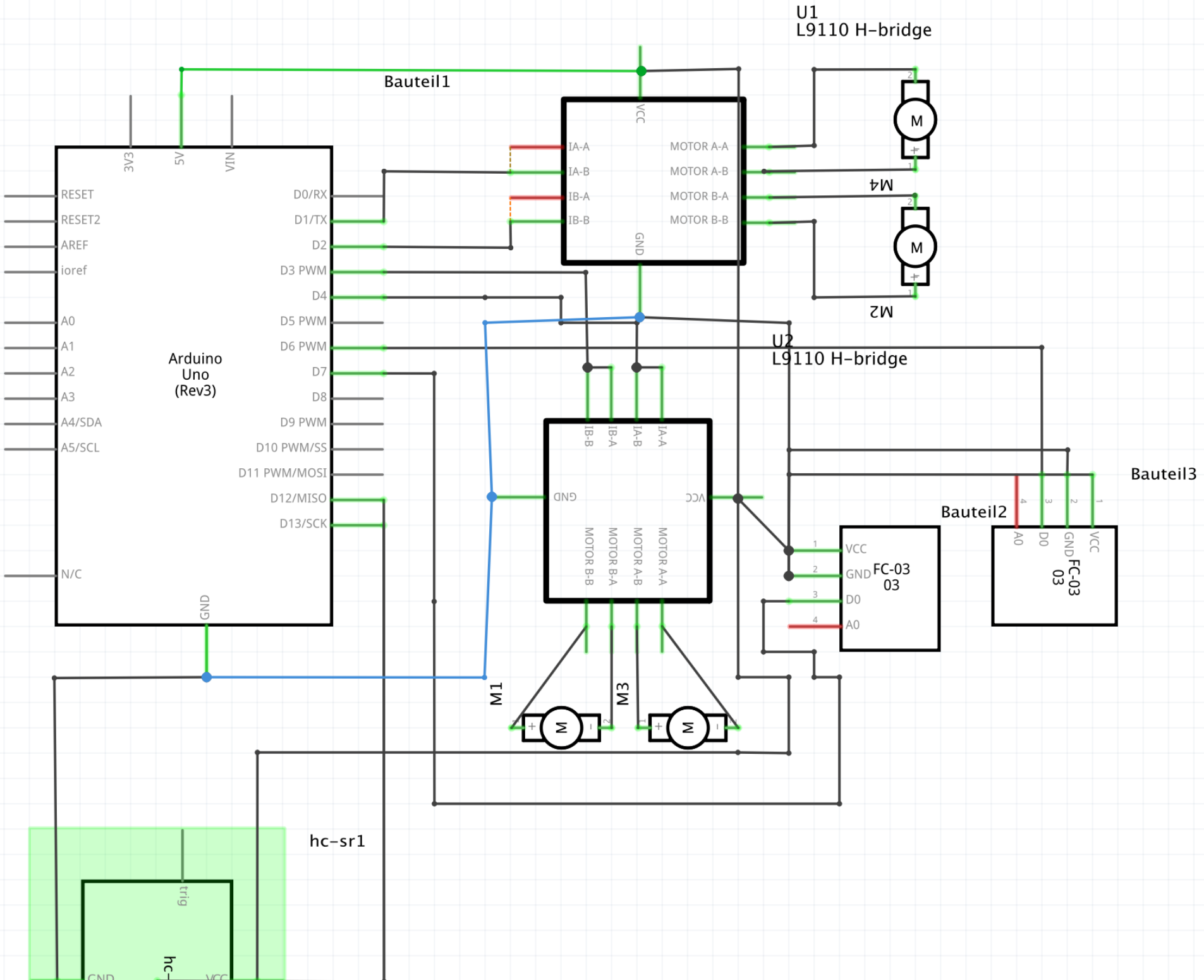
- 1x arduino
- 2x micro controller
- 4x Motoren
- 2x Lichtwellensensor
- 1x Ultraschall sensor
- 4x Räder

Folgende Pins werden dem Mikrocontroller zugeordnet

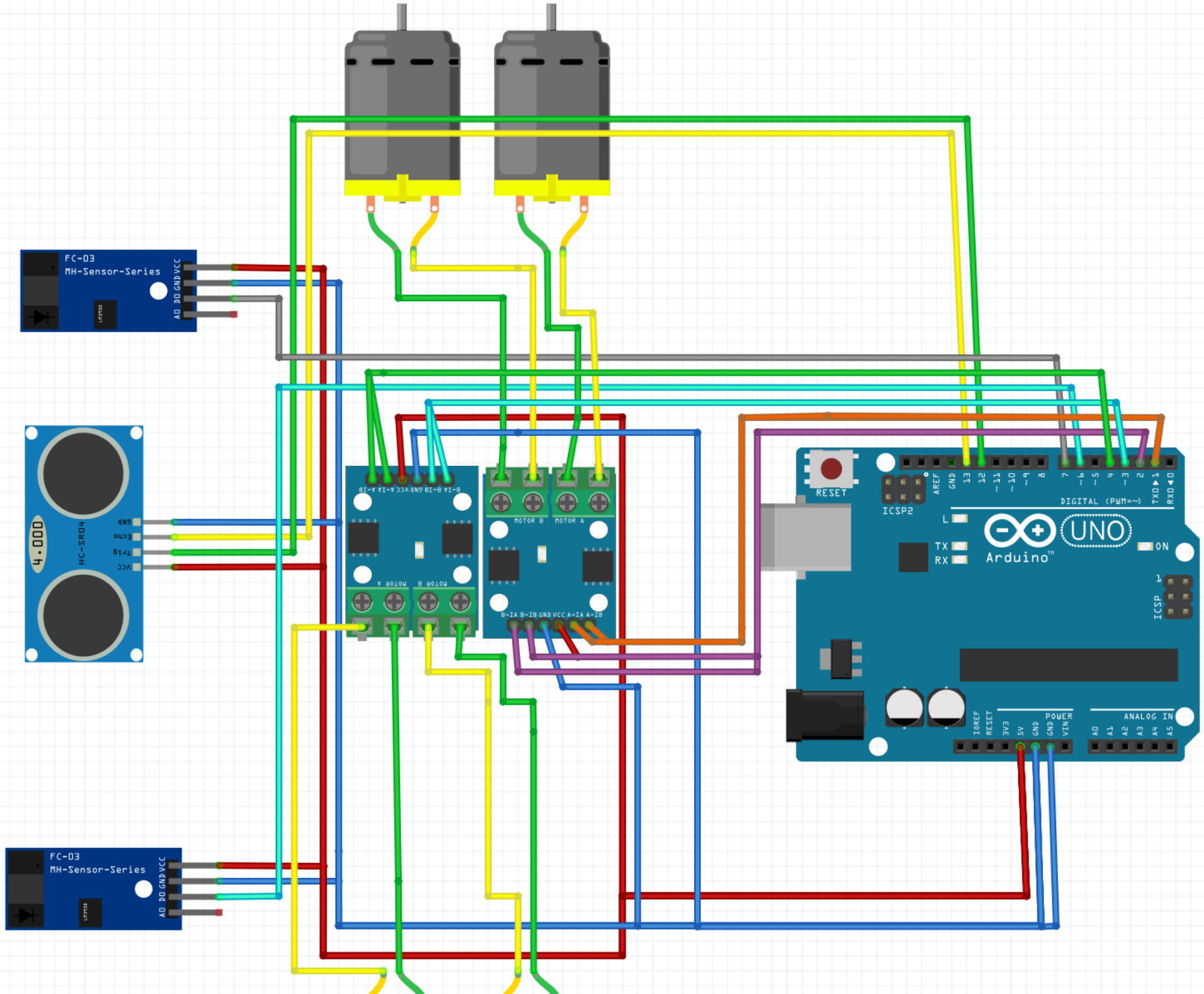
Motor 1 & 3 werden von 3 & 9 Angesprochen
Motor 2 & 4 werden von 10 & 11 Angesprochen

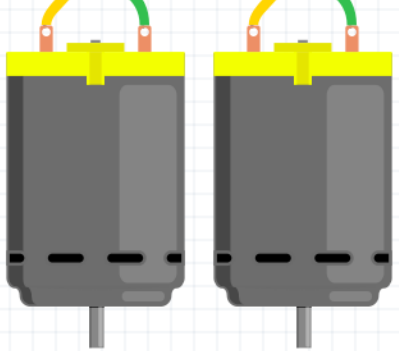
Der Sensor spricht 12 & 13 an

Schaltplan


U1
L9110 H-bridge
Bauteil1
VCC
IA-A
IA-B
IB-A
IB-B
GND
MOTOR A-A
MOTOR A-B
MOTOR B-A
MOTOR B-B
M
M4
M2
3V3
5V
VIN
RESET
RESET2
AREF
ioref
A0
A1
A2
A3
A4/SDA
A5/SCL
N/C
Arduino
Uno
(Rev3)
D0/RX
D1/TX
D2
D3 PWM
D4
D5 PWM
D6 PWM
D7
D8
D9 PWM
D10 PWM/SS
D11 PWM/MOSI
D12/MISO
D13/SCK
GND
U2
L9110 H-bridge
Bauteil3
Bauteil2
VCC
GND
D0
A0
FC-03
03
M1
M3
hc-sr1
trig
hc-
GND
VCC

Layout BREADBOARD


FC-03
MH-Sensor-Series
AO DO GND VCC
HC-SR04
GND
Echo
Trig
VCC
MOTOR B
MOTOR A
B-IA B-IB GND VCC A-IA A-IB
RESET
ICSP2
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX0 ▶1
RX0 ◀0
DIGITAL (PWM=~)
L
TX
RX
UNO
Arduino
ON
ICSP
IOREF
RESET
3V3
5V
GND
GND
VIN
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5



Programmablaufplan

```
   +-----------------------+
   |        Start          |
   +-----------------------+
            |
            v
  +---------------------+
  | Initialisiere Motoren |
  +---------------------+
            |
            v
 +-----------------------+
 | Messung der Distanz     |
 +-----------------------+
            |
            v
 +----------------------------+
 | Distanz < 20 cm?           | -- Nein --> [Fahre vorwärts]
 +----------------------------+
            |
         Ja v
+--------------------------+
| Stoppe alle Motoren        |
```

```
+---------------------------+
           |
           v
  +---------------------+
  | Fahre rückwärts      |
  +---------------------+
           |
           v
  +---------------------+
  | Fahre nach links     |
  +---------------------+
           |
           v
 +-----------------------------+
 | Ist das Hindernis entfernt? |
 +-----------------------------+
           |
        Ja v                        Nein v
  +---------------------+      +----------------------+
  | Drehe nach rechts    |      | Fahre weiter rückwärts|
  +---------------------+      +----------------------+
           |                             |
           v                             v
  +---------------------+      +----------------------+
  | Fahre vorwärts       |      | Stoppe alle Motoren   |
  +---------------------+      +----------------------+
           |
           v
  +---------------------+
  | Stoppe alle Motoren  |
  +---------------------+
           |
           v
  +---------------------+
  | Wiederhole           |
  +---------------------+
```

Programmcode

main.cpp

```
#include <Arduino.h>

#include "CMotorControls.h"

CMotor *m1 = nullptr;
CMotor *m2 = nullptr;
CMotor *m3 = nullptr;
CMotor *m4 = nullptr;
CMotorControls *motorControls;

// Distanzmesser Pins
const int trigPin = 12; // Trigger Pin
const int echoPin = 13; // Echo Pin

// Getter für die Sensor Daten
long getDistance() {
digitalWrite(trigPin, LOW);
delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

long duration = pulseIn(echoPin, HIGH);
long distance = (duration * 0.0343) / 2; // 0.0343 cm/µs

return distance;
}

void setup() {
Serial.begin(9600);
Serial.println("Initialisiere Ports");

// Motoren Parallelschaltung
m1 = new CMotor(8, 9); // Motor auf Port 8 und 9
m2 = new CMotor(8, 9); // Motor auf Port 8 und 9
m3 = new CMotor(10, 11); // Motor auf Port 10 und 11
```

```
m4 = new CMotor(10, 11); // Motor auf Port 10 und 11
motorControls = new CMotorControls(m1, m2, m3, m4); // Gruppiert alle Motoren in ein Objekt

// Setze die Pins für den Ultraschallsensor
pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);
}

void loop() {

// Aktuelle distanz
long distance = getDistance();
Serial.print("Distanz: " + String(distance) + "cm \n");
delay(500)

if (distance < 20) { // Wenn ein Hindernis näher als 20 cm ist
Serial.println("Notbremsung!");
motorControls->move(ECarDirection::NONE);
delay(500);

Serial.println("Fahre zurück");
motorControls->move(ECarDirection::BACKWARD);
delay(3000);

Serial.println("Drehe nach links");
motorControls->move(ECarDirection::LEFT);
delay(3000);

// Wenn das Hindernis weiter als 20 cm ist, nach rechts drehen und vorwärts fahren

if (distance > 20) {
Serial.println("Drehe nach rechts");
motorControls->move(ECarDirection::RIGHT);
delay(3000);

Serial.println("Fahre vorwärts");
motorControls->move(ECarDirection::FORWARD);
delay(3000);

} else {
Serial.println("Noch zu nah, fahre weiter zurück");
motorControls->move(ECarDirection::BACKWARD);
delay(3000);
```

```
 }

} else {
Serial.println("Kein Hindernis erkannt, fahre vorwärts");
motorControls->move(ECarDirection::FORWARD);
 }
}
```

cmotor.cpp

```
#include "CMotor.h"

#include <Arduino.h>

CMotor::CMotor(int p1, int p2) : pin1(p1), pin2(p2) {
pinMode(pin1, OUTPUT);
pinMode(pin2, OUTPUT);
}

void CMotor::rotate(EDirection dir) {
switch (dir) {

case EDirection::FORWARD:
digitalWrite(pin1, HIGH); // Motor rotates forward
digitalWrite(pin2, LOW);
break;

case EDirection::BACKWARD:
digitalWrite(pin1, LOW); // Motor rotates backward
digitalWrite(pin2, HIGH);
break;

case EDirection::NONE:
Serial.println("Motor gestoppt");
digitalWrite(pin1, LOW); // Motor stops (both pins LOW)
digitalWrite(pin2, LOW);
break;
 }
}
```

Cmotorcontrol.cpp

```
#include "CMotorControls.h"

#include <Arduino.h>

CMotorControls::CMotorControls(CMotor *m1, CMotor *m2, CMotor *m3, CMotor *m4) {
motors[0] = m1;
motors[1] = m2;
motors[2] = m3;
motors[3] = m4;
}

void CMotorControls::move(ECarDirection dir) {
switch (dir) {

case ECarDirection::FORWARD:
motors[0]->rotate(EDirection::FORWARD);
Serial.println("Motor1 Vorwärts");
motors[1]->rotate(EDirection::FORWARD);
Serial.println("Motor2 Vorwärts");
motors[2]->rotate(EDirection::FORWARD);
Serial.println("Motor3 Vorwärts");
motors[3]->rotate(EDirection::FORWARD);
Serial.println("Motor4 Vorwärts");
break;

case ECarDirection::BACKWARD:
motors[0]->rotate(EDirection::BACKWARD);
Serial.println("Motor1 Rückwärts");
motors[1]->rotate(EDirection::BACKWARD);
Serial.println("Motor2 Rückwärts");
motors[2]->rotate(EDirection::BACKWARD);
Serial.println("Motor3 Rückwärts");
motors[3]->rotate(EDirection::BACKWARD);
Serial.println("Motor4 Rückwärts");
break;

case ECarDirection::RIGHT:
motors[0]->rotate(EDirection::BACKWARD);
Serial.println("Motor1 Rückwärts");
motors[1]->rotate(EDirection::BACKWARD);
Serial.println("Motor2 Rückwärts");
motors[2]->rotate(EDirection::FORWARD);
```

```
Serial.println("Motor3 Vorwärts");
motors[3]->rotate(EDirection::FORWARD);
Serial.println("Motor4 Vorwärts");
break;

case ECarDirection::LEFT:
motors[0]->rotate(EDirection::FORWARD);
Serial.println("Motor1 Vorwärts");
motors[1]->rotate(EDirection::FORWARD);
Serial.println("Motor2 Vorwärts");
motors[2]->rotate(EDirection::BACKWARD);
Serial.println("Motor3 Rückwärts");
motors[3]->rotate(EDirection::BACKWARD);
Serial.println("Motor4 Rückwärts");
break;

case ECarDirection::NONE: {
motors[0]->rotate(EDirection::NONE);
motors[1]->rotate(EDirection::NONE);
motors[2]->rotate(EDirection::NONE);
motors[3]->rotate(EDirection::NONE);
break;
  }
 }
}
```

cmotor.h

```
// CMotor.h

#ifndef CMOTOR_H

#define CMOTOR_H

enum class EDirection {
FORWARD,
BACKWARD,
NONE
};

class CMotor {
private:
```

```
int pin1, pin2;

public:
CMotor(int p1, int p2);
void rotate(EDirection dir);
};



#endif // CMOTOR_H
```

cmotorcontrol.h

```
#ifndef CMOTORCONTROLS_H

#define CMOTORCONTROLS_H

#include "CMotor.h"

// Enum Klasse für richtungen
enum class ECarDirection {
FORWARD,
BACKWARD,
LEFT,
RIGHT,
NONE
};

class CMotorControls {
private:
CMotor* motors[4]; //Array speichert die motor pointer

public:
// Konstruktor für Motoren
CMotorControls(CMotor *m1, CMotor *m2, CMotor *m3, CMotor *m4);

// Movement Methode
void move(ECarDirection dir);
};

#endif
```

Bilder des Projekts